

lab2实验报告

思考题

Thinking 2.1

Thinking 2.1 请根据上述说明，回答问题：在编写的 C 程序中，指针变量中存储的地址 被视为虚拟地址，还是物理地址？MIPS 汇编程序中 lw 和 sw 指令使用的地址被视为虚拟地址，还是物理地址？

- 在编写的C程序中，指针变量中存储的地址被视为虚拟地址。因为C程序在执行时，其地址空间是虚拟的，由操作系统和硬件的内存管理单元（MMU）负责将虚拟地址映射到物理地址。
- MIPS汇编程序中，lw和sw指令通常使用的地址在标准的MIPS架构中是虚拟地址。这是因为在MIPS架构中，默认情况下，处理器运行在用户模式下，执行的是通过虚拟地址访问内存的指令。只有在特定的异常处理或者操作系统内核模式下，处理器才会直接使用物理地址。

Thinking 2.2

Thinking 2.2 请思考下述两个问题：

- 从可重用性的角度，阐述用宏来实现链表的好处。
- 查看实验环境中的 `/usr/include/sys/queue.h`，了解其中单向链表与循环链表的实现，比较它们与本实验中使用的双向链表，分析三者插入与删除操作上的性能差异。
- 好处：
 - 抽象和通用性：宏允许开发者定义通用的链表操作，而不依赖于特定的数据类型。通过使用宏，可以在不同的数据结构中重用相同的链表实现代码，只需简单地将数据类型作为宏参数传递。
 - 类型无关性：宏不是函数，因此它们不受函数参数类型检查的限制。这意味着同一个宏定义可以用于任何类型的链表节点，提高了代码的可重用性。
 - 代码简洁性：宏可以减少重复代码的编写。对于链表的插入、删除等操作，使用宏可以避免为每种数据类型编写重复的函数。
 - 性能优化：宏在编译时展开，没有函数调用的开销，因此在某些情况下可以提供更好的性能。这在性能敏感的操作系统中是一个重要的考虑因素。
 - 易于维护：链表操作集中在宏定义中，如果需要修改链表的操作逻辑，只需修改宏定义，而不需要修改每个使用链表的地方。
- 差异：
 - 对于单向链表，由于它只能获得每一项的后面一项，因此在删除时需要遍历整个链表；同样，如果是在某一项的前面插入，也需要从头开始遍历这个链表。但是如果是“在某一项之后插入”，单项链表可以直接进行该操作。
 - 对于循环链表，因为它仍然是单向的，所以在“删除”、“某一项之前插入”、“某一项之后插入”三个操作的性能和单项链表相同。但是，由于循环链表首尾相连，同时维护了一个指向尾部的指针，因此它可以直接在尾部插入。
 - 对于双向链表，因为它可以直接获得某一项的前后两项，所以无论是“删除”还是“在某一项前或后插入”都可以实现。但是，双向链表没有维护指向尾部的指针，因此无法直接将某一项插入链表尾部，如要实现该操作还需要遍历整个链表。

Thinking 2.3

Thinking 2.3 请阅读 include/queue.h 以及 include/pmap.h, 将 Page_list 的结构梳理清楚, 选择正确的展开结构。

- ```
1 A:
2 struct Page_list{
3 struct {
4 struct {
5 struct Page *le_next;
6 struct Page *le_prev;
7 } pp_link;
8 u_short pp_ref;
9 } * lh_first;
10 }
```
- ```
1 B:
2 struct Page_list{
3     struct {
4         struct {
5             struct Page *le_next;
6             struct Page **le_prev;
7         } pp_link;
8         u_short pp_ref;
9     } lh_first;
10 }
```
- ```
1 C:
2 struct Page_list{
3 struct {
4 struct {
5 struct Page *le_next;
6 struct Page **le_prev;
7 } pp_link;
8 u_short pp_ref;
9 } * lh_first;
10 }
```

正确的展开是C

## Thinking 2.4

Thinking 2.4 请思考下面两个问题:

- 请阅读上面有关 TLB 的描述, 从虚拟内存和多进程操作系统的实现角度, 阐述 ASID 的必要性。
- 请阅读 MIPS 4Kc 文档《MIPS32® 4K™ Processor Core Family Software User's Manual》的 Section 3.3.1 与 Section 3.4, 结合 ASID 段的位数, 说明 4Kc 中可容纳不同的地址空间的最大数量。
- 操作系统会给每一个进程分配一个页表, 每个页表都有自己的虚拟地址空间, 而同一虚拟地址在不同地址空间中通常映射到不同的物理地址。ASID 负责区分当前虚拟地址是在哪个进程中使用, 避免将该虚拟地址映射到错误的物理地址。
- 由下图可知, ASID有8位, 则可以容纳不同的地址空间数量为 $2^8 = 256$

Figure 3-7 show the contents of one of the 16 dual-entries in the JTLB.

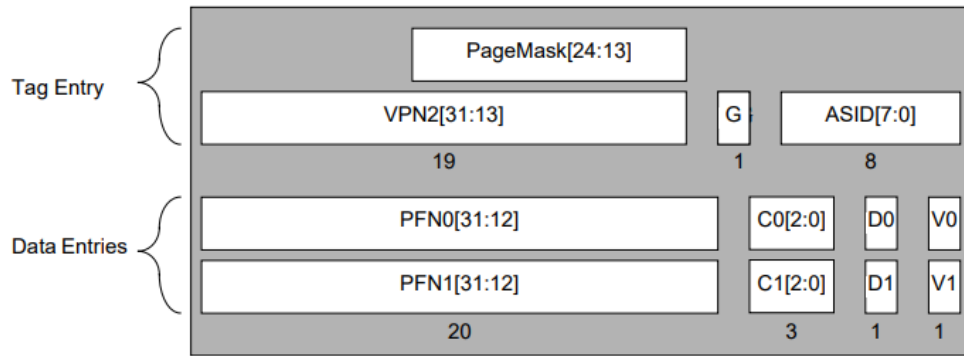


Figure 3-7 JTLB Entry (Tag and Data)

## Thinking 2.5

Thinking 2.5 请回答下述三个问题：

- tlb\_invalidate 和 tlb\_out 的调用关系？
  - 请用一句话概括 tlb\_invalidate 的作用。
  - 逐行解释 tlb\_out 中的汇编代码。
- tlb\_invalidate 函数中调用了 tlb\_out 函数
  - 在页表内容改变后更新 TLB。
- ```
1 LEAF(tlb_out)
2 .set noreorder
3 mfc0    t0, CP0_ENTRYHI # 将EntryHi寄存器的值存入t0寄存器
4 mtc0    a0, CP0_ENTRYHI # 将a0中的值存入EntryHi寄存器
5 nop
6 nop
7 tlbvp # 根据EntryHi在TLB中查找与之对应的表项，并把表项的索引存入Index寄存器
8 nop
9 nop
10 nop
11 mfc0    t1, CP0_INDEX # 将Index寄存器的值存入t1寄存器
12 .set reorder
13 bltz    t1, NO_SUCH_ENTRY # 如果t1值小于零，即没有在TLB中找到EntryHi对应的
    # 表项，则跳转到NO_SUCH_ENTRY标签处
14 nop
15 .set noreorder
16 mtc0    zero, CP0_ENTRYHI # 将EntryHi寄存器赋值为0
17 mtc0    zero, CP0_ENTRYLO0 # 将EntryLo0寄存器赋值为0
18 mtc0    zero, CP0_ENTRYLO1 # 将EntryLo1寄存器赋值为0
19 nop
20 tlbwi # 将EntryHi和EntryLo0寄存器中的值存取Index对应的TLB表项中
21 .set reorder
22
23 NO_SUCH_ENTRY:
24 mtc0    t0, CP0_ENTRYHI # 将t0寄存器中的值存入EntryHi寄存器(恢复到调用该
    # 函数之前的状态)
25 j       ra # 函数返回
```

Thinking 2.6

Thinking 2.6 请结合 Lab2 开始的 CPU 访存流程与下图中的 Lab2 用户函数部分，尝试将函数调用与 CPU 访存流程对应起来，思考函数调用与 CPU 访存流程的关系。

- 函数调用流程：
 - 调用函数：程序执行到函数调用指令时，会暂停当前函数的执行，并将控制权传递给被调用的函数。
 - 保存上下文：在调用函数之前，需要保存当前函数的状态，包括寄存器中的值和返回地址（通常是栈上的一个地址）。
 - 传递参数：将函数的参数传递给被调用的函数，这通常通过寄存器或栈来完成。
 - 执行函数体：控制权转移到被调用的函数，开始执行函数体内的指令。
 - 返回值：函数执行完成后，将结果（如果有的话）返回给调用者，通常是通过寄存器或栈。
 - 恢复上下文：恢复调用函数之前保存的状态，包括寄存器的值和程序计数器（PC）。
 - 继续执行：控制权返回到调用函数，继续执行调用点之后的指令。
- CPU 访存流程：
 - 取指（Fetch）：CPU 从内存中取出下一条指令，并将其放入指令寄存器。
 - 译码（Decode）：CPU 解析指令，确定需要执行的操作和操作数。
 - 执行（Execute）：CPU 执行指令指定的操作，这可能包括算术逻辑运算、数据传输等。
 - 访存（Memory）：如果指令需要访问内存（如加载或存储操作），CPU 会在这个阶段进行内存读写。
 - 写回（Write Back）：将执行结果写回到寄存器或内存。
- 对应关系：
 - 取指对应于函数调用的准备阶段，包括确定函数地址和准备参数。
 - 译码对应于函数调用的参数处理和确定函数体开始执行的位置。
 - 执行对应于函数体内部的指令执行，包括局部变量的声明和计算。
 - 访存对应于函数执行期间可能发生的内存操作，如访问局部变量、全局变量或动态分配的内存。
 - 写回对应于函数执行结果的保存，以及可能的返回值处理。

总的来说，函数调用是高级语言中的一个抽象概念，而CPU访存流程是这一抽象概念在硬件层面的实现。函数调用的每一步都需要CPU通过一系列的访存和计算操作来实现。

练习题

Exercise 2.1

Exercise 2.1 请参考代码注释，补全 `mips_detect_memory` 函数。在实验中，指定了硬件可用内存大小 `memsize`，请你用内存大小 `memsize` 完成总物理 页数 `npage` 的初始化。

```
1 | npage = memsize >> PGSHIFT;
```

这里需要计算页面数量。在 `mmu.h` 中可以发现 `PGSHIFT` 这个宏定义，表示页面大小取以2为底的对数的值，这里为12。意思是页面大小为 $4096 = 2^{12}$ 。`memsize` 右移 `PGSHIFT` 相当于 $memsize / 2^{PGSHIFT}$ ，得到的就是页面数量。

Exercise 2.2

Exercise 2.2 完成 `include/queue.h` 中空缺的函数 `LIST_INSERT_AFTER`。其功能是将一个元素插入到已有元素之后，可以仿照 `LIST_INSERT_BEFORE` 函数来实现。

```
1 | #define LIST_INSERT_AFTER(listelm, elm, field)
    | \
2 |     /* Exercise 2.2 Your code here */
    | \
3 |     do {
    | \
```

```

4      /*将新元素elm的field字段指向listelm元素后面那个元素的地址*/
      \
5      (LIST_NEXT((elm), field)) = (LIST_NEXT((listelm), field));
      \
6      /*如果listelm后面有元素，则需要更新那个元素的le_prev指针，使其指向新元素elm*/
      \
7      if ((LIST_NEXT((listelm), field)) != NULL) {
      \
8          LIST_NEXT((listelm), field)->field.le_prev = &(LIST_NEXT((elm),
field));
      \
9      }
      \
10     /*将listelm的field字段更新为指向新元素elm，这样就将新元素插入到了listelm之后
*/
      \
11     LIST_NEXT((listelm), field) = (elm);
      \
12     /*更新新元素elm的le_prev字段，使其指向listelm的field字段，完成双向链表的链接
*/
      \
13     (elm)->field.le_prev = &(LIST_NEXT((listelm), field));
      \
14     } while (0)
15     /*反斜杠是为了让编译器把整个宏当作一行，do-while是为了确保宏只运行一次，且正确识别分
号*/
16

```

这里需要重点理解链表项结构体的 `le_prev` 字段的含义，它是指向上一个元素的 `le_next` 字段的指针，是一个指针的指针。

Exercise 2.3

Exercise 2.3 完成 `page_init` 函数。请按照函数中的注释提示，完成上述操作。此外，这里也给出一些提示：1. 使用链表初始化宏 `LIST_INIT`。2. 将 `freemem` 按照 `PAGE_SIZE` 进行对齐（使用 `ROUND` 宏为 `freemem` 赋值）。3. 将 `freemem` 以下页面对应的页控制块中的 `pp_ref` 标为 1。4. 将其它页面对应的页控制块中的 `pp_ref` 标为 0 并使用 `LIST_INSERT_HEAD` 将其插入空闲链表。

```

1 void page_init(void) {
2     /* Step 1: Initialize page_free_list. */
3     /* Hint: Use macro `LIST_INIT` defined in include/queue.h. */
4     /* Exercise 2.3: Your code here. (1/4) */
5
6     // 使用链表初始化宏 LIST_INIT
7     LIST_INIT(&page_free_list);
8
9     /* Step 2: Align `freemem` up to multiple of PAGE_SIZE. */
10    /* Exercise 2.3: Your code here. (2/4) */
11
12    // 将 freemem 按照 PAGE_SIZE 进行对齐（使用 ROUND 宏为 freemem 赋值）。
13    freemem = ROUND(freemem, PAGE_SIZE);
14
15    /* Step 3: Mark all memory below `freemem` as used (set `pp_ref` to 1)
*/
16    /* Exercise 2.3: Your code here. (3/4) */
17
18    // 将 freemem 以下页面对应的页控制块中的 pp_ref 标为 1。
19    int size = PADDR(freemem) / PAGE_SIZE;

```

```

20     int i;
21     for (i = 0; i < size; ++i) {
22         pages[i].pp_ref = 1;
23     }
24
25     /* Step 4: Mark the other memory as free. */
26     /* Exercise 2.3: Your code here. (4/4) */
27
28     // 将其它页面对应的页控制块中的 pp_ref 标为 0 并使用 LIST_INSERT_HEAD 将其插入
    空闲链表。
29     for (i = size; i < npage; ++i) {
30         pages[i].pp_ref = 0;
31         LIST_INSERT_HEAD(&page_free_list, pages + i, pp_link);
32     }
33 }

```

Exercise 2.4

Exercise 2.4 完成 `page_alloc` 函数。在 `page_init` 函数运行完毕后，在 MOS 中如果想申请存储空间，都需要通过这个函数来申请分配。该函数的逻辑简单来说，可以表述为：1. 如果空闲链表没有可用页，返回异常返回值。2. 如果空闲链表有可用的页，取出链表头部的一页；初始化后，将该页对应的页控制块的地址放到调用者指定的地方。填空时，你可能需要使用链表宏 `LIST_EMPTY` 与函数 `page2kva`。

```

1  int page_alloc(struct Page **new) {
2      /* Step 1: Get a page from free memory. If fails, return the error
    code.*/
3      struct Page *pp;
4      /* Exercise 2.4: Your code here. (1/2) */
5
6      // 如果空闲链表没有可用页，返回异常返回值。这里也可以用LIST_EMPTY来判断
7      pp = LIST_FIRST(&page_free_list);
8      if (pp == NULL) {
9          return -E_NO_MEM;
10     }
11
12     LIST_REMOVE(pp, pp_link);
13
14     /* Step 2: Initialize this page with zero.
15      * Hint: use `memset`. */
16     /* Exercise 2.4: Your code here. (2/2) */
17
18     // 如果空闲链表有可用的页，取出链表头部的一页；初始化后，将该页对应的页控制块的地址放
    到调用者指定的地方。
19     // memset负责初始化（把内容清零），page2kva负责将一个指向物理页的指针转换为其对应
    的虚拟地址
20     memset((void *)page2kva(pp), 0, PAGE_SIZE);
21
22     *new = pp;
23     return 0;
24 }

```

Exercise 2.5

Exercise 2.5 完成 `page_free` 函数。提示：使用链表宏 `LIST_INSERT_HEAD`，将页结构体插入空闲页结构体链表。

```

1 void page_free(struct Page *pp) {
2     assert(pp->pp_ref == 0);
3     /* Just insert it into 'page_free_list'. */
4     /* Exercise 2.5: Your code here. */
5
6     LIST_INSERT_HEAD(&page_free_list, pp, pp_link);
7
8 }

```

直接调用宏 LIST_INSERT_HEAD 即可

Exercise 2.6

Exercise 2.6 完成 pgdir_walk 函数。该函数的作用是：给定一个虚拟地址，在给定的页目录中查找这个虚拟地址对应的页表项，将其地址写入 *ppte。

- 如果这一虚拟地址对应的二级页表存在，则设置 *ppte 为这一页表项的地址；
- 如果这一虚拟地址对应的二级页表不存在（即这一虚拟地址对应的页目录项无效），则当 create 不为 0 时先创建二级页表再查找页表项，为 0 时则将 *ppte 设置为空指针。注意，这里可能会在页目录项无效且 create 为真时，使用 page_alloc 创建一个页表，此时应维护申请得到的物理页的 pp_ref 字段。

```

1 static int pgdir_walk(Pde *pgdir, u_long va, int create, Pte **ppte) {
2     Pde *pgdir_entry;
3     struct Page *pp;
4
5     /* Step 1: Get the corresponding page directory entry. */
6     /* Exercise 2.6: Your code here. (1/3) */
7
8     // 计算偏移量：页表基值 + 一级页表号。这里的PDX是一个宏定义，获取逻辑地址va的31-22
    位
9     pgdir_entry = pgdir + PDX(va);
10
11     /* Step 2: If the corresponding page table is not existent (valid)
    then:
12     *   * If parameter `create` is set, create one. Set the permission
    bits 'PTE_C_CACHEABLE |
13     *       PTE_V' for this new page in the page directory. If failed to
    allocate a new page (out
14     *       of memory), return the error.
15     *   * Otherwise, assign NULL to '*ppte' and return 0.
16     */
17
18     /* Exercise 2.6: Your code here. (2/3) */
19
20     // 如果这一虚拟地址对应的二级页表不存在（即这一虚拟地址对应的页目录项无效），则当
    create 不为 0 时先创建二级页表再查找页表项，为 0 时则将 *ppte 设置为空指针。
21     if (!((*pgdir_entry) & PTE_V)) {
22         if (create) {
23             try(page_alloc(&pp));
24             *pgdir_entry = page2pa(pp);
25             *pgdir_entry = (*pgdir_entry) | PTE_C_CACHEABLE | PTE_V;
26             pp->pp_ref++;
27         } else {
28             *ppte = 0;
29             return 0;

```

```

30     }
31 }
32
33
34 /* Step 3: Assign the kernel virtual address of the page table entry to
35  *ppte'. */
36 /* Exercise 2.6: Your code here. (3/3) */
37
38 // 如果这一虚拟地址对应的二级页表存在，则设置 *ppte 为这一页表项的地址
39 *ppte = (Pte *)KADDR(PTE_ADDR(*pgdir_entryp)) + PTX(va);
40
41 return 0;
42 }

```

Exercise 2.7

```

1  int page_insert(Pde *pgdir, u_int asid, struct Page *pp, u_long va, u_int
2  perm) {
3
4      /* Step 1: Get corresponding page table entry. */
5      pgdir_walk(pgdir, va, 0, &pte);
6
7      if (pte && (*pte & PTE_V)) {
8          if (pa2page(*pte) != pp) {
9              page_remove(pgdir, asid, va);
10         } else {
11             tlb_invalidate(asid, va);
12             *pte = page2pa(pp) | perm | PTE_C_CACHEABLE | PTE_V;
13             return 0;
14         }
15     }
16
17     /* Step 2: Flush TLB with 'tlb_invalidate'. */
18     /* Exercise 2.7: Your code here. (1/3) */
19
20     // 使 TLB 中与虚拟地址 va 相关的条目无效，因为下面的操作可能会更改页表项。
21     tlb_invalidate(asid, va);
22
23     /* Step 3: Re-get or create the page table entry. */
24     /* If failed to create, return the error. */
25     /* Exercise 2.7: Your code here. (2/3) */
26
27     // 再次调用 pgdir_walk，这次第三个参数为 1，表示如果页表项不存在，则创建一个新的页
28     // 表项
29     try(pgdir_walk(pgdir, va, 1, &pte));
30
31     /* Step 4: Insert the page to the page table entry with 'perm |
32      PTE_C_CACHEABLE | PTE_V'
33      * and increase its 'pp_ref'. */
34     /* Exercise 2.7: Your code here. (3/3) */
35
36     // 将物理页的物理地址与权限位组合，并设置页表项。增加物理页的引用计数 pp_ref，表示现
37     // 在有一个新的虚拟地址映射到该物理页。
38     *pte = page2pa(pp) | perm | PTE_C_CACHEABLE | PTE_V;
39     pp->pp_ref++;
40 }

```

```

38     return 0;
39 }

```

Exercise 2.8

Exercise 2.8 完成 kern/tlb_asm.S 中的 tlb_out 函数。该函数根据传入的参数 (TLB 的 Key) 找到对应的 TLB 表项, 并将其清空。具体来说, 需要在两个位置插入两条指令, 其中一个位置为 tlbtp, 另一个位置为 tlbwi。因流水线设计架构原因, tlbtp 指令的前后都应各插入一个 nop 以解决数据冒险。

```

1  LEAF(tlb_out)
2  .set noreorder
3      mfc0    t0, CP0_ENTRYHI
4      mtc0    a0, CP0_ENTRYHI
5      nop
6      /* Step 1: Use 'tlbtp' to probe TLB entry */
7      /* Exercise 2.8: Your code here. (1/2) */
8
9      tlbtp
10     nop
11
12     nop
13     /* Step 2: Fetch the probe result from CP0.Index */
14     mfc0    t1, CP0_INDEX
15     .set reorder
16     bltz    t1, NO_SUCH_ENTRY
17     .set noreorder
18     mtc0    zero, CP0_ENTRYHI
19     mtc0    zero, CP0_ENTRYLO0
20     mtc0    zero, CP0_ENTRYLO1
21     nop
22     /* Step 3: Use 'tlbwi' to write CP0.EntryHi/Lo into TLB at CP0.Index */
23     /* Exercise 2.8: Your code here. (2/2) */
24
25     tlbwi
26
27     .set reorder

```

需要注意题目提示: “因流水线设计架构原因, tlbtp 指令的前后都应各插入一个 nop 以解决数据冒险。”

Exercise 2.9

Exercise 2.9 完成 kern/tlbex.c 中的 _do_tlb_refill 函数。

```

1  void _do_tlb_refill(u_long *pentrylo, u_int va, u_int asid) {
2      tlb_invalidate(asid, va);
3      Pte *ppte;
4      /* Hints:
5       *   Invoke 'page_lookup' repeatedly in a loop to find the page table
6       *   entry '*ppte'
7       *   associated with the virtual address 'va' in the current address
8       *   space 'cur_pgdir'.
9       *   **while** 'page_lookup' returns 'NULL', indicating that the '*ppte'
10      could not be found,

```

```

9      * allocate a new page using 'passive_alloc' until 'page_lookup'
succeeds.
10     */
11
12     /* Exercise 2.9: Your code here. */
13
14     // 尝试在循环中调用 'page_lookup' 以查找虚拟地址 va
15     // 在当前进程页表中对应的页表项 '*ppte'
16     // 如果 'page_lookup' 返回 'NULL', 表明 '*ppte' 找不到, 使用 'passive_alloc'
17     // 为 va 所在的虚拟页面分配物理页面,
18     // 直至 'page_lookup' 返回不为 'NULL' 则退出循环。
19     while (page_lookup(cur_pgdir, va, &ppte) == NULL) {
20         passive_alloc(va, cur_pgdir, asid);
21     }
22
23     ppte = (Pte *)((u_long)ppte & ~0x7);
24     pentrylo[0] = ppte[0] >> 6;
25     pentrylo[1] = ppte[1] >> 6;
26 }

```

Exercise 2.10

Exercise 2.10 完成 kern/tlb_asm.S 中的 do_tlb_refill 函数。

```

1  do_tlb_refill_call:
2      addi    sp, sp, -24 /* Allocate stack for arguments(3), return
value(2), and return address(1) */
3      sw      ra, 20(sp) /* [sp + 20] - [sp + 23] store the return address */
4      addi    a0, sp, 12 /* [sp + 12] - [sp + 19] store the return value */
5      jal     _do_tlb_refill /* (Pte *, u_int, u_int) [sp + 0] - [sp + 11]
reserved for 3 args */
6      lw      a0, 12(sp) /* Return value 0 - Even page table entry */
7      lw      a1, 16(sp) /* Return value 1 - Odd page table entry */
8      lw      ra, 20(sp) /* Return address */
9      addi    sp, sp, 24 /* Deallocate stack */
10     mtc0    a0, CP0_ENTRYLO0 /* Even page table entry */
11     mtc0    a1, CP0_ENTRYLO1 /* Odd page table entry */
12     nop
13     /* Hint: use 'tlbwr' to write CP0.EntryHi/Lo into a random tlb entry.
*/
14     /* Exercise 2.10: Your code here. */
15
16     /*tlbwr: 将 EntryHi 与 EntryLo0、EntryLo1 的数据随机写到一个 TLB 表项中（此处
使用 Random 寄存器来“随机”指定表项，Random 寄存器本质上是一个不停运行的循环计数器）。*/
17     tlbwr
18
19     jr      ra
20
21 END(do_tlb_refill)
22

```

难点

一、二级页表管理下的虚实地址转换

地址和页表项构成

逻辑地址 = 一级页表号 | 二级页表号 | 页内偏移

- 假设为 $10 + 10 + 12 = 32$ 位，即页大小为 $2^{12} = 0x1000$

物理地址 = 物理页框号 | 页内偏移

- 假设为 $20 + 12 = 32$ 位

页表项 = 物理页框号 | 标志位

- 假设为 $20 + 12 = 32$ 位

访问流程

访问一级页表

从PCB中获取一级页表所在的物理页框号，假设为0x1，在 $0x1 * 0x1000$ 的位置根据一级页表号访问得到一级页表项。

访问二级页表

根据一级页表项中的物理页框号（高20位），假设为0xe7， $0xe7 * 0x1000$ 的位置根据二级页表号访问得到二级页表项。

计算物理地址

根据二级页表项中的物理页框号（高20位），接上页内偏移，得到的就是物理地址。

二、页结构体和链表的理解

Page 结构体包含两部分——一个是"数据域" (pp_ref), 另一个是"指针域" (pp_link)。

```
1 struct Page{
2     u_short pp_ref;
3     struct {
4         struct Page *le_next;
5         struct Page **le_prec;
6     } pp_link;
7 };
```

其中，pp_ref 代表着该物理内存块被引用的次数，即有多少个虚拟地址映射到该物理内存块。

pp_link 中包含两个指针变量——le_next 和 le_prec，前者是指向后一个Page结构体的指针，后者是指向前一个Page的 le_next 的指针。

特别需要注意：“指针的指针”

三、TLB快表的操作

TLB的组成、相关汇编指令，对TLB的操作对我来说都是全新的内容，需要多阅读指导书熟悉。